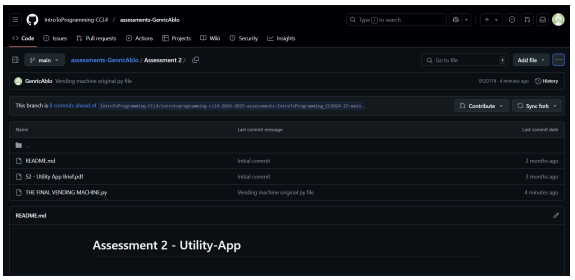
Intro to Programming

Assessment 2: Utility App

Student's Name	Ablo, Genric Shawn B. Ablo
Roll No	2023-215
Github Repository Name	https://github.com/IntroToProgramming-CCL4/assessments-GenricAblo/tree/0520174fb2d1f445e75989e172af79db944a375b/Assessment%202
Github Repository Link	https://github.com/IntroToProgramming-CCL4/assessments-GenricAblo/tree/main/Assessment%202
Repository Screen Shot	

The Development Document

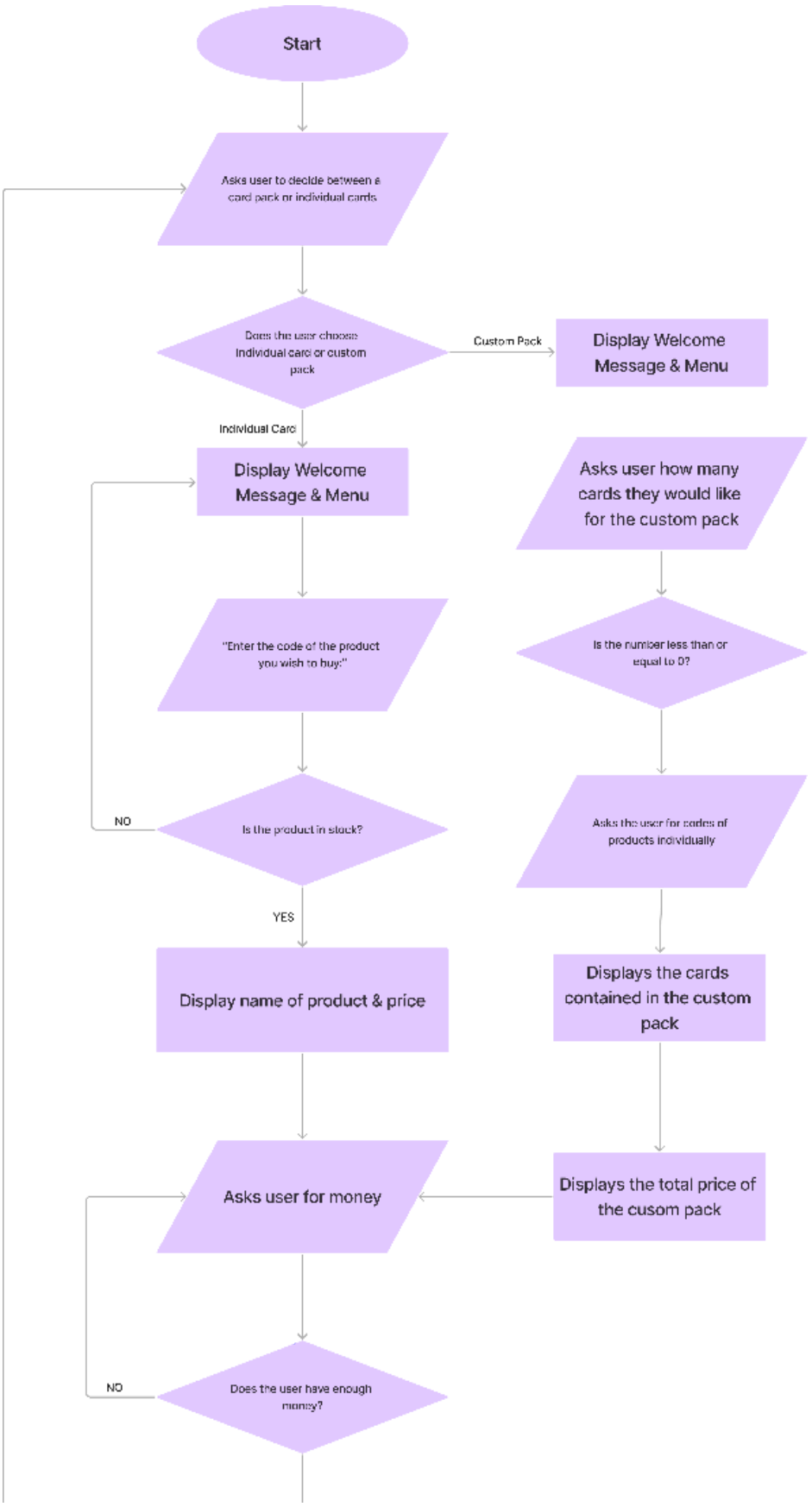
Specification

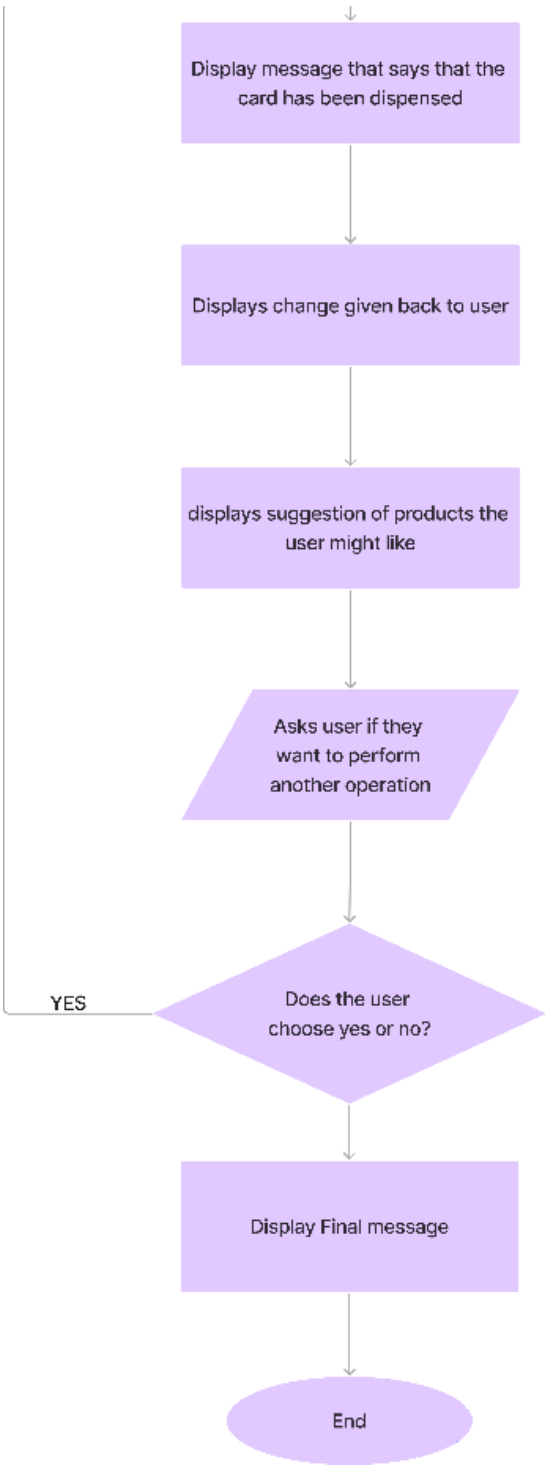
I was tasked to build a vending machine with the use of Python. I created a vending machine that dispensed collectible Pokemon cards and Baseball cards. I included numerous features such as:

- A set number of codes that corresponds to different products
- A menu that displays necessary information
- Restocking Cards
- Stock system
- Different ways to purchase multiple products at once
- Suggested items at checkout
- Change
- Personalized messages for multiple scenarios
- Error handling in case of possible user-created mistakes

System Flowchart

This flowchart shows how the front-end of the code interacts with the user, this visualizes the interactions the user had to make to use the vending machine


Start
Asks user to decide between a card pack or individual cards
Does the user choose individual card or custom pack
Custom Pack
Display Welcome Message & Menu
Individual Card
Display Welcome Message & Menu
"Enter the code of the product you wish to buy:"
Is the product in stock?
NO
YES
Display name of product & price
Asks user for money
Does the user have enough money?
NO
Asks user how many cards they would like for the custom pack
Is the number less than or equal to 0?
Asks the user for codes of products individually
Displays the cards contained in the custom pack
Displays the total price of the cusom pack



Technical Description & Walkthrough

(7-minute walkthrough)

https://youtu.be/leK6Apn1cPM

Critical Reflection

Creating the basics of a vending machine was fairly simple, only minor issues like missing colons and indents(still happens throughout). The major issues only began when I wanted to include new features, it was difficult to include them into the basic line of code as is. So what I did was to define functions for all these features, then use all these functions inside a main function where everything is executed. I had a lot of fun researching all these things like class and the random module.

I was really downplaying my struggle in the beginning, I started to really over think even the smallest things that happen inside the line of programming. However, as I started coding, I began to understand how certain things work. One of the biggest hurdles I had was how to store the products' information in a way that I can extract them in the future. When that happened, I had to search online ways to do this. I learned what Object- Oriented Programming was and used that for my code.

I had a good time creating this vending machine. It felt really rewarding when I was able to iron out all the issues I could find. There are many ways I could have optimized my code and made it better, I could have added more unique products, added more features to help with user satisfaction, and more things like that. But I was really proud of how it turned out.

References:

GeeksforGeeks (2016). *Python OOPs Concepts*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python-oops-concepts/.

W3Schools (2019). *Python Classes*. [online] W3schools.com. Available at: https://www.w3schools.com/python/python_classes.asp.

Appendix

```
import random



class VendingMachine:

    def __init__(self):

        # Items available in the vending machine categorized

        self.categories = {

            "Pokemon Cards": {

                "A1": {"name": "Pikachu Card", "price": 5.00, "stock":
random.randint(0, 20)},

                "A2": {"name": "Rare Pikachu Card", "price": 25.00,
"stock": random.randint(0, 12)},

                "A3": {"name": "Ultra Rare Pikachu Card", "price":
200.00, "stock": random.randint(0, 5)},



                "B1": {"name": "Charizard Card", "price": 5.00,
"stock": random.randint(0, 30)},

                "B2": {"name": "Rare Charizard Card", "price": 30.00,
"stock": random.randint(0, 10)},

                "B3": {"name": "Ultra Rare Charizard Card", "price":
450.00, "stock": random.randint(0, 5)},
```

```
                "C1": {"name": "Bulbasaur Card", "price": 6.00,
"stock": random.randint(0, 20)},

                "C2": {"name": "Rare Bulbasaur Card", "price": 26.73,
"stock": random.randint(0, 10)},

                "C3": {"name": "Ultra Rare Bulbasaur Card", "price":
230.00, "stock": random.randint(0, 3)},



                "D1": {"name": "Legendary Box", "price": 60.00,
"stock": random.randint(0, 5)},

            },

            "Baseball Cards": {

                "E1": {"name": "Babe Ruth Card", "price": 9.00,
"stock": random.randint(0, 25)},

                "E2": {"name": "Rare Babe Ruth Card", "price": 38.00,
"stock": random.randint(0, 10)},

                "E3": {"name": "Ultra Rare Babe Ruth Card", "price":
235.00, "stock": random.randint(0, 10)},



                "F1": {"name": "Mickey Mantle Card", "price": 6.00,
"stock": random.randint(0, 20)},

                "F2": {"name": "Rare Mickey Mantle Card", "price":
30.00, "stock": random.randint(0, 12)},

                "F3": {"name": "Ultra Rare Mickey Mantle Card",
"price": 230.00, "stock": random.randint(0, 5)},
```

```
                "G1": {"name": "Jackie Robinson Card", "price": 7.00,
"stock": random.randint(0, 5)},

                "G2": {"name": "Rare Jackie Robinson Card", "price":
34.00, "stock": random.randint(0, 5)},

                "G3": {"name": "Ultra Rare Jackie Robinson Card",
"price": 450.00, "stock": random.randint(0, 5)},



                "H1": {"name": "Legendary Card pack", "price": 400.00,
"stock": random.randint(0, 5)},

            },

        }



    def create_custom_pack(self):

        # Allows the user to create a custom pack

        print("\nCreate your custom card pack! Here are the available
cards:")

        # Display available cards across all categories

        available_cards = {}

        for category, items in self.categories.items():

            print(f"\n{category}:")

            for code, details in items.items():
```

```
                if details["stock"] > 0:  # Only show cards in stock

                    available_cards[code] = details

                    print(f'{code}: ${details['price'] : <5}-
{details['name'] : ^35} | Stock: {details['stock'] }')



        if not available_cards:

            print("\nNo cards are currently available to create a
custom pack.")

            return



        # Let the user select a pack size

        try:

            pack_size = int(input("\nHow many cards would you like to
add to your custom pack? "))

            if pack_size <= 0:

                print("❌Invalid pack size. Please enter a positive
number.")

                return

        except ValueError:

            print("❌Invalid input. Please enter a number.")

            return
```

```
        # Collect user selections

        selected_cards = []

        total_cost = 0

        for i in range(pack_size):

            card_code = input(f"Enter the code of card {i + 1}:
").strip().upper()

            if card_code in available_cards:

                selected_card = available_cards[card_code]

                if selected_card["stock"] > 0:

                    selected_cards.append(selected_card)

                    total_cost += selected_card["price"]

                    selected_card["stock"] -= 1  # Deduct stock
immediately

                else:

                    print(f"Sorry, {selected_card['name']} is out of
stock.")

            else:

                print(f"❌Invalid card code: {card_code}")



        if not selected_cards:

            #user did not choose a card available in the vending
machine. loops it
```

```
            print("You did not select any valid cards.")
            return


        print("\nYour custom pack contains:")
        #displays what is inside the custom pack created by the user
        for card in selected_cards:
            print(f"  - {card['name']} | Price: ${card['price']:.2f}")


        print(f"\nTotal cost of your custom pack: ${total_cost:.2f}")


        # Process payment
        try:
            payment = float(input("Enter your payment amount: "))
            if payment < total_cost:
                print("Insufficient payment. Transaction cancelled.")
                # Restore stock for the selected cards
                for card in selected_cards:
                    card["stock"] += 1
                return
        except ValueError:
```

```
            print("❌Invalid payment. Please enter a valid number.")
            # Restore stock for the selected cards
            for card in selected_cards:
                card["stock"] += 1
            return


        change = round(payment - total_cost, 2)
        print("\nThank you for your purchase! Enjoy your custom pack!")
        if change > 0:
            print(f"Your change is: ${change:.2f}")









        print("")
    def display_menu(self):
        #Displays menu
        print("\nWelcome to the Card Vending Machine!\nHere is the list
of available cards:")
        print("")
        for category, items in self.categories.items():
```

```
            print(f"\n{category}:")

            for code, details in items.items():

                print(f'{code}: $ {details['price'] : <5}-
{details['name'] : ^35} | Stock: {details['stock'] }')



    def get_item(self, code):

        #Function designed to extract an item from the code of an item
given by the user

        for category in self.categories.values():

            if code.upper() in category:

                return category[code.upper()]

        return None



    def process_purchase(self, code, payment):

        #Transaction process

        item = self.get_item(code)

        if not item:

            return "❌Invalid code. Please try again.", payment



        if item["stock"] == 0:

            #
```

```
            return f"Sorry, {item['name']} is out of stock.", payment



        if payment < item["price"]:

            return f"Insufficient funds. {item['name']} costs
${item['price']:.2f}.", payment




        item["stock"] -= 1

        change = round(payment - item["price"], 2)

        return f"Dispensing {item['name']}! Enjoy your card!", change



    def suggest_items(self, category_name):

        if category_name not in self.categories:

            return



        print("You might also like:")

        suggestions = [

            details["name"]

            for code, details in self.categories[category_name].items()

            if details["stock"] > 0
```

```
        ]

        if suggestions:

            print(", ".join(suggestions))

        else:

            print("No similar items available.")









def main():

    machine = VendingMachine()



    while True :

            choice = input("Would you like to:\n  P = Purchase
Individual Cards\n  C = Create a Custom Pack(Best for purchasing
multiple products)\nEnter your choice: ").strip().upper()



            if choice == 'P':
```

```
                machine.display_menu()

                # Ask for user selection for individual cards

                code = input("\nEnter the code of the card you wish to
buy: ")

                item = machine.get_item(code)

                if not item:

                    print("❌Invalid code. Please try again.")

                    continue



                # Determine the category for suggestions

                category_name = next(

                    (category for category, items in
machine.categories.items() if code.upper() in items),

                    None,

                )



                # Check stock

                if item["stock"] == 0:

                    print(f"Sorry, {item['name']} is out of stock.")

                    continue
```

```
                # Ask for payment

                print(f"The price of {item['name']} is
${item['price']:.2f}.")

                try:

                    payment = float(input("Enter the amount of money
you are inserting: "))

                    if payment <= 0:

                        print("Invalid payment. Please insert a valid
amount.")

                        continue

                except ValueError:

                    print("Invalid input. Please enter a number.")

                    continue



                # Process purchase

                message, change = machine.process_purchase(code,
payment)

                print(message)



                if isinstance(change, float):

                    print(f"Your change is: ${change:.2f}")
```

```
                # Suggest similar items

                machine.suggest_items(category_name)

                more = input('\nWould you like to perform another
operaion? (yes/no): ').strip().lower()

                if more != "no":

                    continue

                else:

                    print("Thank you for using this vending machine.
Have a nice day!")

                    quit()

            elif choice == 'C':

                # Custom pack creation

                machine.create_custom_pack()

                more = input('\nWould you like to perform another
operaion? (yes/no): ').strip().lower()

                if more != "no":

                    continue

                else:

                    print("Thank you for using this vending machine.
Have a nice day!")

                    quit()
```

```
            else:
                print("❌Invalid choice. Please enter 'P' or 'C'.")
            continue


if __name__ == '__main__':
    main()
```